

private 继承”意味，在考虑过所有其他方案之后，如果仍然认为 private 继承是“表现程序内两个 classes 之间的关系”的最佳办法，这才用它。

请记住

- Private 继承意味 is-implemented-in-terms of（根据某物实现出）。它通常比复合（composition）的级别低。但是当 derived class 需要访问 protected base class 的成员，或需要重新定义继承而来的 virtual 函数时，这么设计是合理的。
- 和复合（composition）不同，private 继承可以造成 empty base 最优化。这对致力于“对象尺寸最小化”的程序库开发者而言，可能很重要。

条款 40：明智而审慎地使用多重继承

Use multiple inheritance judiciously.

一旦涉及多重继承（multiple inheritance; **MI**），C++ 社群便分为两个基本阵营。其中之一认为如果单一继承（single inheritance; **SI**）是好的，多重继承一定更好。另一派阵营则主张，单一继承是好的，但多重继承不值得拥有（或使用）。本条款的主要目的是带领大家了解多重继承的两个观点。

最先需要认清的一件事是，当 **MI** 进入设计景框，程序有可能从一个以上的 base classes 继承相同名称（如函数、typedef 等等）。那会导致较多的歧义（ambiguity）机会。例如：

```
class BorrowableItem {           //图书馆允许你借某些东西
public:
    void checkOut( );             //离开时进行检查
    ...
};

class ElectronicGadget {
private:
    bool checkOut( ) const;       //执行自我检测，返回是否测试成功
    ...
};

class MP3Player:                 //注意这里的多重继承
    public BorrowableItem,        //（某些图书馆愿意借出 MP3 播放器）
    public ElectronicGadget
{ ... };                         //这里，class 的定义不是我们的关心重点

MP3Player mp;
mp.checkOut( );                  //歧义！调用的是哪个 checkOut？
```

注意此例之中对 `checkOut` 的调用是歧义（模棱两可）的，即使两个函数之中只有一个可取用（`BorrowableItem` 内的 `checkOut` 是 `public`，`ElectronicGadget` 内的却是 `private`）。这与 C++ 用来解析（`resolving`）重载函数调用的规则相符：在看到是否有个函数可取用之前，C++ 首先确认这个函数对此调用之言是最佳匹配。找出最佳匹配函数后才检验其可取用性。本例的两个 `checkOuts` 有相同的匹配程度（译注：因此才造成歧义），没有所谓最佳匹配。因此 `ElectronicGadget::checkOut` 的可取用性也就从未被编译器审查。

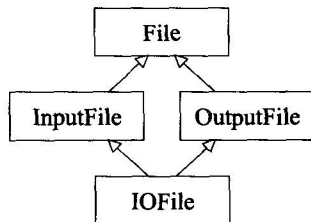
为了解决这个歧义，你必须明白指出你要调用哪一个 `base class` 内的函数：

```
mp.BorrowableItem::checkOut();           //哎呀，原来是这个 checkOut...
```

你当然也可以尝试明确调用 `ElectronicGadget::checkOut`，但然后你会获得一个“尝试调用 `private` 成员函数”的错误。

多重继承的意思是继承一个以上的 `base classes`，但这些 `base classes` 并不常在继承体系中又有更高级的 `base classes`，因为那会导致要命的“钻石型多重继承”：

```
class File { ... };  
class InputFile: public File { ... };  
class OutputFile: public File { ... };  
class IOFile: public InputFile,  
              public OutputFile  
{ ... };
```

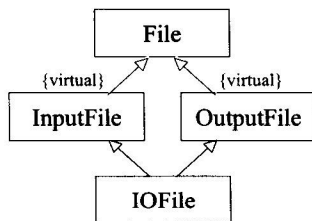


任何时候如果你有一个继承体系而其中某个 `base class` 和某个 `derived class` 之间有一条以上的相通路线（就像上述的 `File` 和 `IOFile` 之间有两条路径，分别穿越 `InputFile` 和 `OutputFile`），你就必须面对这样一个问题：是否打算让 `base class` 内的成员变量经由每一条路径被复制？假设 `File class` 有个成员变量 `fileName`，那么 `IOFile` 内该有多少笔这个名称的数据呢？从某个角度说，`IOFile` 从其每一个 `base class` 继承一份，所以其对象内应该有两份 `fileName` 成员变量。但从另一个角度说，简单的逻辑告诉我们，`IOFile` 对象只该有一个文件名称，所以它继承自两个 `base classes` 而来的 `fileName` 不该重复。

C++ 在这场辩论中并没有倾斜立场；两个方案它都支持——虽然其缺省做法是执行复制（也就是上一段所说的第一个做法）。如果那不是你要的，你必须令那个带有此数据的 `class`（也就是 `File`）成为一个 *virtual base class*。为了这样做，你必

须令所有直接继承自它的 classes 采用“virtual 继承”：

```
class File { ... };  
class InputFile: virtual public File { ... };  
class OutputFile: virtual public File { ... };  
class IOFile: public InputFile,  
               public OutputFile  
{ ... };
```



C++ 标准程序库内含一个多重继承体系，其结构就如右图那样，只不过其 classes 其实是 class templates，名称分别是 `basic_ios`，`basic_istream`，`basic_ostream` 和 `basic_iostream`，而非这里的 `File`，`InputFile`，`OutputFile` 和 `IOFile`。

从正确行为的观点看，`public` 继承应该总是 `virtual`。如果这是唯一一个观点，规则很简单：任何时候当你使用 `public` 继承，请改用 `virtual public` 继承。但是，啊呀，正确性并不是唯一观点。为避免继承得来的成员变量重复，编译器必须提供若干幕后戏法，而其后果是：使用 `virtual` 继承的那些 classes 所产生的对象往往比使用 `non-virtual` 继承的兄弟们体积大，访问 `virtual base classes` 的成员变量时，也比访问 `non-virtual base classes` 的成员变量速度慢。种种细节因编译器不同而异，但基本重点很清楚：你得为 `virtual` 继承付出代价。

`virtual` 继承的成本还包括其他方面。支配“`virtual base classes` 初始化”的规则比起 `non-virtual bases` 的情况远为复杂且不直观。`virtual base` 的初始化责任是由继承体系中的最低层（*most derived*）class 负责，这暗示（1）classes 若派生自 `virtual bases` 而需要初始化，必须认知其 `virtual bases`——不论那些 `bases` 距离多远，（2）当一个新的 `derived class` 加入继承体系中，它必须承担其 `virtual bases`（不论直接或间接）的初始化责任。

我对 `virtual base classes`（亦相当于对 `virtual` 继承）的忠告很简单。第一，非必要不使用 `virtual bases`。平常请使用 `non-virtual` 继承。第二，如果你必须使用 `virtual base classes`，尽可能避免在其中放置数据。这么一来你就不需担心这些 classes 身上的初始化（和赋值）所带来的诡异事情了。Java 和 .NET 的 `Interfaces` 值得注意，它在许多方面兼容于 C++ 的 `virtual base classes`，而且也不允许含有任何数据。

现在让我们看看下面这个用来塑模“人”的 C++ `Interface class`（见条款 31）：

```
class IPerson {
public:
    virtual ~IPerson();
    virtual std::string name() const = 0;
    virtual std::string birthDate() const = 0;
};
```

IPerson 的客户必须以 IPerson 的 **pointers** 和 **references** 来编写程序, 因为抽象 **classes** 无法被实体化创建对象。为了创建一些可被当做 IPerson 来使用的对象, IPerson 的客户使用 **factory functions** (工厂函数, 见条款 31) 将“派生自 IPerson 的具象 classes”实体化:

```
//factory function (工厂函数), 根据一个独一无二的数据库 ID 创建一个 Person 对象。
//条款 18 告诉你为什么返回类型不是原始指针。
std::tr1::shared_ptr<IPerson> makePerson(DatabaseID personIdentifier);

//这个函数从使用者手上取得一个数据库 ID
DatabaseID askUserForDatabaseID();

DatabaseID id(askUserForDatabaseID());
std::tr1::shared_ptr<IPerson> pp(makePerson(id));
//创建一个对象支持 IPerson 接口。
... //藉由 IPerson 成员函数处理 *pp。
```

但是 makePerson 如何创建对象并返回一个指针指向它呢? 无疑地一定有某些派生自 IPerson 的具象 class, 在其中 makePerson 可以创建对象。

假设这个 class 名为 CPerson。就像具象 class 一样, CPerson 必须提供“继承自 IPerson”的 **pure virtual** 函数的实现代码。我们可以从无到有写出这些东西, 但更好的是利用既有组件, 后者做了大部分或所有必要事情。例如, 假设有个既有的数据库相关 class, 名为 PersonInfo, 提供 CPerson 所需要的实质东西:

```
class PersonInfo {
public:
    explicit PersonInfo(DatabaseID pid);
    virtual ~PersonInfo();
    virtual const char* theName() const;
    virtual const char* theBirthDate() const;
    ...
private:
    virtual const char* valueDelimOpen() const; //详下
    virtual const char* valueDelimClose() const;
    ...
};
```

你可以说这是个旧式 class，因为其成员函数返回 `const char*` 而不是 `string` 对象。尽管如此，如果鞋子合脚，干嘛不穿它？这个 class 的成员函数的名称已经暗示我们其结果有可能很令人满意。

你会发现，`PersonInfo` 被设计用来协助以各种格式打印数据库字段，每个字段值的起始点和结束点以特殊字符串为界。缺省的头尾界限符号是方括号（中括号），所以（例如）字段值 `"Ring-tailed Lemur"` 将被格式化为：

```
[Ring-tailed Lemur]
```

但由于方括号并非放之四海人人喜爱的界限符号，所以两个 `virtual` 函数 `valueDelimOpen` 和 `valueDelimClose` 允许 `derived classes` 设定它们自己的头尾界限符号。`PersonInfo` 成员函数将调用这些 `virtual` 函数，把适当的界限符号添加到它们的返回值上。以 `PersonInfo::theName` 为例，代码看起来像这样：

```
const char* PersonInfo::valueDelimOpen() const
{
    return "[";           //缺省的起始符号
}

const char* PersonInfo::valueDelimClose() const
{
    return "]";           //缺省的结尾符号
}

const char* PersonInfo::theName() const
{
    //保留缓冲区给返回值使用；由于缓冲区是 static，因此会被自动初始化为“全部是 0”
    static char value[Max_Formatted_Field_Value_Length];
    //写入起始符号
    std::strcpy(value, valueDelimOpen());
    现在，将 value 内的字符串添附到这个对象的 name 成员变量中
    （小心，避免缓冲区超限）
    //写入结尾符号
    std::strcat(value, valueDelimClose());
    return value;
}
```

或许有人质疑 `PersonInfo::theName` 的老旧设计（特别是它竟然使用固定大小的 `static` 缓冲区，那将充斥超限问题和线程问题，见条款 21），但是不妨暂时把这样的疑问放两旁，把以下焦点摆中间：`theName` 调用 `valueDelimOpen` 产生字符串起始符号，然后产生 `name` 值，然后调用 `valueDelimClose`。由于 `valueDelimOpen`

和 `valueDelimClose` 都是 `virtual` 函数, `theName` 返回的结果不仅取决于 `PersonInfo` 也取决于从 `PersonInfo` 派生下去的 `classes`。

身为 `CPerson` 实现者, 这是个好消息, 因为仔细阅读 `IPerson` 文档后, 你发现 `name` 和 `birthDate` 两函数必须返回未经装饰 (不带起始符号和结尾符号) 的值。也就是说如果有人名为 `Homer`, 调用其 `name` 函数理应获得 `"Homer"` 而不是 `"[Homer]"`。

`CPerson` 和 `PersonInfo` 的关系是, `PersonInfo` 刚好有若干函数可帮助 `CPerson` 比较容易实现出来。就这样。它们的关系因此是 `is-implemented-in-terms-of` (根据某物实现出), 而我们知道这种关系可以两种技术实现: 复合 (见条款 38) 和 `private` 继承 (见条款 39)。条款 39 指出复合通常是较受欢迎的做法, 但如果需要重新定义 `virtual` 函数, 那么继承是必要的。本例之中 `CPerson` 需要重新定义 `valueDelimOpen` 和 `valueDelimClose`, 所以单纯的复合无法应付。最直接的解法就是令 `CPerson` 以 `private` 形式继承 `PersonInfo`, 虽然条款 39 也说过, 只要多加一点工作, `CPerson` 也可以结合 “复合 + 继承” 技术以求有效重新定义 `PersonInfo` 的 `virtual` 函数。此处我将使用 `private` 继承。

但 `CPerson` 也必须实现 `IPerson` 接口, 那需得以 `public` 继承才能完成。这导致多重继承的一个通情达理的应用: 将 “`public` 继承自某接口” 和 “`private` 继承自某实现” 结合在一起:

```
class IPerson {                                     //这个 class 指出需实现的接口
public:
    virtual ~IPerson();
    virtual std::string name() const = 0;
    virtual std::string birthDate() const = 0;
};

class DatabaseID { ... };                           //稍后被使用; 细节不重要。

class PersonInfo {                                  //这个 class 有若干有用函数,
public:                                              //可用以实现 IPerson 接口。
    explicit PersonInfo(DatabaseID pid);
    virtual ~PersonInfo();
    virtual const char* theName() const;
    virtual const char* theBirthDate() const;
    virtual const char* valueDelimOpen() const;
    virtual const char* valueDelimClose() const;
    ...
};
```

```

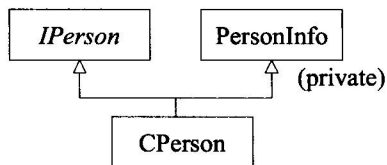
class CPerson: public IPerson, private PersonInfo { //注意, 多重继承
public:
    explicit CPerson(DatabaseID pid): PersonInfo(pid) { }
    virtual std::string name() const           //实现必要的 IPerson 成员函数
    { return PersonInfo::theName(); }

    virtual std::string birthDate() const      //实现必要的 IPerson 成员函数
    { return PersonInfo::theBirthDate(); }

private:
    const char* valueDelimOpen() const { return ""; }           //重新定义
    const char* valueDelimClose() const { return ""; }          //继承而来的
                                                                // virtual
                                                                //"界限函数"
};

```

在 UML 图中这个设计看起来像这样:



这个例子告诉我们, 多重继承也有它的合理用途。

故事结束前, 请容我说, 多重继承只是面向对象工具箱里的一个工具而已。和单一继承比较, 它通常比较复杂, 使用上也比较难以理解, 所以如果你有个单一继承的设计方案, 而它大约等价于一个多重继承设计方案, 那么单一继承设计方案几乎一定比较受欢迎。如果你唯一能够提出的设计方案涉及多重继承, 你应该更努力想一想——几乎可以说一定会有某些方案让单一继承行得通。然而多重继承有时候的确是完成任务之最简洁、最易维护、最合理的做法, 果真如此就别害怕使用它。只要确定, 你的确是在明智而审慎的情况下使用它。

请记住

- 多重继承比单一继承复杂。它可能导致新的歧义性, 以及对 virtual 继承的需要。
- virtual 继承会增加大小、速度、初始化 (及赋值) 复杂度等等成本。如果 virtual base classes 不带任何数据, 将是最具实用价值的情况。
- 多重继承的确有正当用途。其中一个情节涉及 “public 继承某个 Interface class” 和 “private 继承某个协助实现的 class” 的两相组合。